

Here, the implementation of `traverse` is using `map2` and `unit`, and we've seen that `map2` can be implemented in terms of `flatMap`:

```
def map2[A,B,C](ma: F[A], mb: F[B])(f: (A,B) => C): F[C] =
  flatMap(ma)(a => map(mb)(b => f(a,b)))
```

What you may not have noticed is that a large number of the useful combinators on `Monad` can be defined using only `unit` and `map2`. The `traverse` combinator is one example—it doesn't call `flatMap` directly and is therefore agnostic to whether `map2` is primitive or derived. Furthermore, for many data types, `map2` can be implemented directly, without using `flatMap`.

All this suggests a variation on `Monad`—the `Monad` interface has `flatMap` and `unit` as primitives, and derives `map2`, but we can obtain a *different* abstraction by letting `unit` and `map2` be the primitives. We'll see that this new abstraction, called an *applicative functor*, is less powerful than a monad, but we'll also see that limitations come with benefits.

12.2 The Applicative trait

Applicative functors can be captured by a new interface, `Applicative`, in which `map2` and `unit` are primitives.

Listing 12.1 Creating the Applicative interface

```
trait Applicative[F[_]] extends Functor[F] {
  // primitive combinators
  def map2[A,B,C](fa: F[A], fb: F[B])(f: (A, B) => C): F[C]
  def unit[A](a: => A): F[A]

  // derived combinators
  def map[B](fa: F[A])(f: A => B): F[B] =
    map2(fa, unit(()))((a, _) => f(a))
  def traverse[A,B](as: List[A])(f: A => F[B]): F[List[B]] =
    as.foldRight(unit(List[B]()))((a, fbs) => map2(f(a), fbs)(_ :: _))
}
```

We can implement `map` in terms of `unit` and `map2`.

Definition of `traverse` is identical.

Recall `()` is the sole value of type `Unit`, so `unit(())` is calling `unit` with the dummy value `()`.

This establishes that *all applicatives are functors*. We implement `map` in terms of `map2` and `unit`, as we've done before for particular data types. The implementation is suggestive of laws for `Applicative` that we'll examine later, since we expect this implementation of `map` to preserve structure as dictated by the `Functor` laws.

Note that the implementation of `traverse` is unchanged. We can similarly move other combinators into `Applicative` that don't depend directly on `flatMap` or `join`.



EXERCISE 12.1

Transplant the implementations of as many combinators as you can from `Monad` to `Applicative`, using only `map2` and `unit`, or methods implemented in terms of them.

```
def sequence[A](fas: List[F[A]]): F[List[A]]
def replicateM[A](n: Int, fa: F[A]): F[List[A]]
def product[A,B](fa: F[A], fb: F[B]): F[(A,B)]
```

EXERCISE 12.2

Hard: The name *applicative* comes from the fact that we can formulate the Applicative interface using an alternate set of primitives, `unit` and the function `apply`, rather than `unit` and `map2`. Show that this formulation is equivalent in expressiveness by defining `map2` and `map` in terms of `unit` and `apply`. Also establish that `apply` can be implemented in terms of `map2` and `unit`.

```
trait Applicative[F[_]] extends Functor[F] {

  def apply[A,B](fab: F[A => B])(fa: F[A]): F[B]
  def unit[A](a: => A): F[A]

  def map[A,B](fa: F[A])(f: A => B): F[B]
  def map2[A,B,C](fa: F[A], fb: F[B])(f: (A,B) => C): F[C]
}
```

Define in terms of `map2` and `unit`.

Define in terms of `apply` and `unit`.

EXERCISE 12.3

The `apply` method is useful for implementing `map3`, `map4`, and so on, and the pattern is straightforward. Implement `map3` and `map4` using only `unit`, `apply`, and the curried method available on functions.¹

```
def map3[A,B,C,D](fa: F[A],
                  fb: F[B],
                  fc: F[C])(f: (A, B, C) => D): F[D]

def map4[A,B,C,D,E](fa: F[A],
                    fb: F[B],
                    fc: F[C],
                    fd: F[D])(f: (A, B, C, D) => E): F[E]
```

Furthermore, we can now make `Monad[F]` a subtype of `Applicative[F]` by providing the default implementation of `map2` in terms of `flatMap`. This tells us that *all monads are applicative functors*, and we don't need to provide separate `Applicative` instances for all our data types that are already monads.

¹ Recall that given `f: (A,B) => C`, `f.curried` has type `A => B => C`. A curried method exists for functions of any arity in Scala.